

Fondamenti di programmazione

Canale A-L - Prof. Maurizio Mancini

AA 26-27

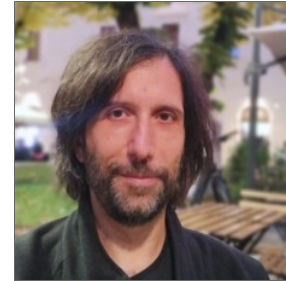
Lezione 1



# Conosciamoci: prof. Maurizio Mancini

## Interessi di ricerca

- HCI
- VR
- CV
- MI



Sito del corso: su Google Classroom

- installatevi **Google Classroom** sullo smartphone
- USATE L'EMAIL ISTITUZIONALE  
**COGNOME.MATRICOLA@STUDENTI.UNIROMA1.IT**
- oppure **seguite le istruzioni su Sapienza**
- iscrivetevi al corso

**e47ucyaq**



# Coordinamento tra i corsi

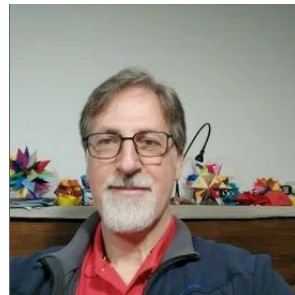
I 4 corsi di programmazione Python corrono in parallelo,  
con stesso programma e stesso esame (siete 500+ studenti!)

## TELEDIDATTICA



Sterbini

Programming  
(ACSAI)



Sterbini

Esercitazioni  
AL



De Sensi

Canale  
MZ



Melonio

Esercitazioni  
MZ



Melonio

Progr  
Lab



Spo

Usate i TUTOR che conoscerete alle esercitazioni!  
Sono una risorsa essenziale e vi possono aiutare con installazioni e nella  
programmazione.

## Lezioni

- mercoledì' dalle 11 alle 13 aula 1L
- venerdì' dalle 10 alle 13 aula 3DL
- aula **1L**  
**Via del Castro**  
**Laurenziano 7a**
- aula **3DL**  
**Via De Lollis ~20(?)**

NO live-streaming ma le lezioni vengono registrate e messe su classroom

**Ricevimento su Zoom**  
**previo appuntamento**



## Esercitazioni

- mercoledì dalle 8 alle 11
- nei lab 15 e 17  
a **via Tiburtina 205**
- col prof. Daniele De Sensi e i tutor





# Esame

**Prova in laboratorio** con 5-7 esercizi (facili e meno facili) da svolgere in 2-3 ore

**NO esame orale**

Min 18 punti, Max 32 (>30 = lode)

## Simulazioni d'esame

Svolgeremo 5 simulazioni d'esame **in laboratorio negli orari di esercitazione** (sugli argomenti svolti fino a quel momento), che vi danno un **BONUS di 0.5 punti ciascuna (se le superate)** da sommare al voto d'esame

- sarà **necessaria la prenotazione** almeno 1 settimana prima per poter organizzare i turni in laboratorio  
(la pagina di prenotazioni sarà pubblicata su classroom)

## Calendario e argomenti delle SYM

|                                                                                                                                                                                                                       |                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 21 ottobre                                                                                                                                                                                                            | strutture dati Python, funzioni e cicli                   |
| 11 novembre                                                                                                                                                                                                           | file                                                      |
| 25 novembre                                                                                                                                                                                                           | immagini                                                  |
| 9 dicembre                                                                                                                                                                                                            | ricorsione                                                |
| <b>16 dicembre</b>                                                                                                                                                                                                    | <b>simulazione completa con tutti i tipi di esercizio</b> |
| <ul style="list-style-type: none"><li>• <b>l'ultima, se superata, può valere (+ bonus) come voto d'esame ed essere verbalizzata a gennaio</b></li><li>• Chi si da molto da fare può finire prima di Natale!</li></ul> |                                                           |

## Software da installare

Usiamo la distribuzione Python **Anaconda**, per Linux/Mac/Windows

Su **classroom** trovate l'elenco delle librerie Python da installare per eseguire i test degli esercizi sul vostro PC

Per programmare potete usare un qualsiasi ambiente di sviluppo: **Spyder**, **Pycharm** o **Visual Studio** o altri

Nelle esercitazioni ed agli esami usiamo una **macchina virtuale Linux** che vi potete scaricare.

Contiene Python e i 3 ambienti di sviluppo e le librerie.

Nel file README trovate le istruzioni per installarla sul vostro PC

# Argomenti del corso

Per chi non ne sa nulla (prime 2 settimane)

- Linguaggio Python
- Tipi di dati, variabili, strutture dati
- Costrutti di controllo: **if-then-else**, **while**, **for**, **match**, **funzioni**

Per chi ne sa già qualcosa (resto del corso)

- Analisi e suddivisione di problemi in problemi più semplici:
  - decomposizione sequenziale, condizionale, iterativa, funzionale, ricorsiva
  - algoritmi di ordinamento
- Immagini, disegno, riconoscimento, filtri
- Cenni di programmazione Object Oriented
- Ricorsione, alberi binari, alberi n-ari, alberi di gioco, combinazioni e permutazioni

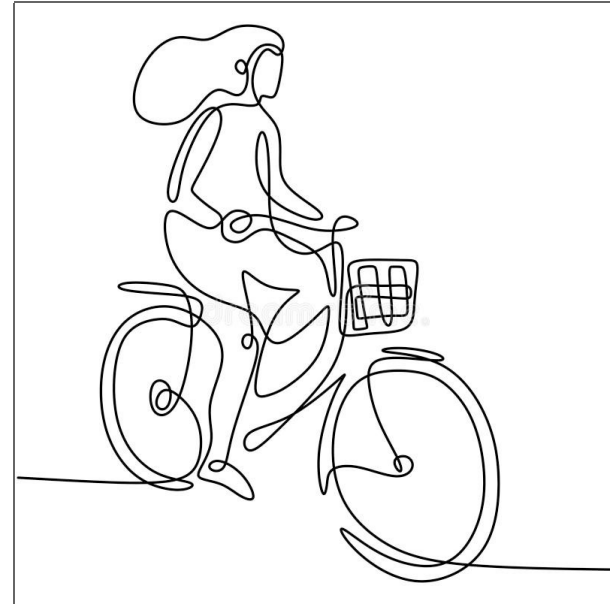


OBIETTIVO DEL CORSO:

FARVI PROGRAMMARE IN PRATICA

Si impara a programmare solo con l'esercizio

Come per imparare ad andare in  
bicicletta



O imparare a suonare la chitarra  
E IMPROVVISARE



Questo **NON è un corso teorico** di algoritmi e non servono pre-requisiti particolari.

Unico pre-requisito: avere un metodo scientifico e un minimo di inglese

Serve tanta chiarezza e «pulizia mentale» e **molta esercitazione**

Il corso e l'esame sono percepiti come **molto difficili** dai vostri colleghi.

- è necessario fare **moltissima pratica** perché l'esame è pratico  
(il calcolatore decide il voto, non noi)
- Si fa **MALE** l'esame, se si pensa  
«mi ricordo l'algoritmo MergeSort e se me lo chiedono mi va bene»
- Si fa **BENE** l'esame se dato un problema nuovo mai visto:
  - si rimane freddi (avete visto problemi simili in aula e a casa)
  - lo si analizza [su carta] per spezzarlo in **sottoproblemi più semplici**
  - si scrive una implementazione quasi corretta
  - **si testano le parti e le si debugga in maniera efficiente**

## COME TOPPARE L'ESAME?

1. Creare **PROGRAMMONI INTRICATI** rende il debugging lentissimo (e ci si perde parecchio tempo all'esame)
2. Creare e manipolare le **IMMAGINI** è molto laborioso (e ci si perde parecchio tempo all'esame)
3. Risolvere i problemi **RICORSIVAMENTE** è concettualmente difficile (e se non vi viene l'ispirazione siete fregati)
4. **Saltare la seconda parte** del corso (che è quella più succosa)

QUINDI:

- SPEZZATE IL PROGRAMMA IN FUNZIONI PICCOLE
- SEGUITE ANCHE LA SECONDA META' DEL CORSO

CHE E' LA PIU' IMPORTANTE (IMMAGINI e RICORSIONE)

A proposito di CHATGPT e simili

Potete usare tool generativi per aiutarvi nello scrivere codice

Ad esempio per spezzare un problema in problemi più semplici

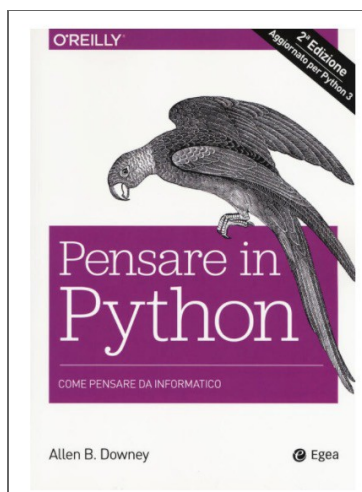
MA **dovete essere in grado di capire** SE quello che CHATGPT scrive:

- **risolve veramente** il vostro problema?
- inserisce **piccoli errori logici** difficili da riconoscere?

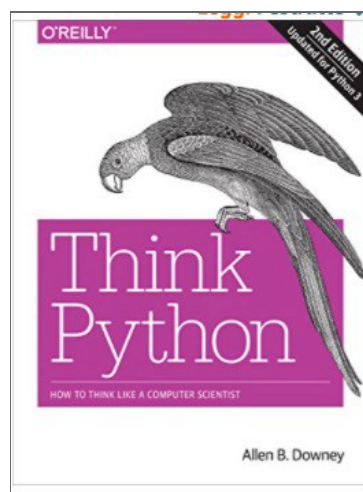
I Large Language Models programmicchiano decentemente ma in almeno 1/3 dei casi producono errori.

NOTA: all'esame sarete sconnessi e **non avrete a disposizione CHATGPT**

qualche libro utile



gratis in  
download



gratis in  
download



Fino al capitolo  
11



Fino al capitolo  
12

## Altre risorse utili

- **Python tutorial:** <https://docs.python.org/3/tutorial>
- **Documentazione ufficiale:** <https://docs.python.org/3>
- **"Bignami" del Python:** <https://gto76.github.io/python-cheatsheet>
- **Più avanzati**
  - <https://python-reference.readthedocs.io>
  - <https://book.pythontips.com>

MA PRIMA UN PO' DI STORIA

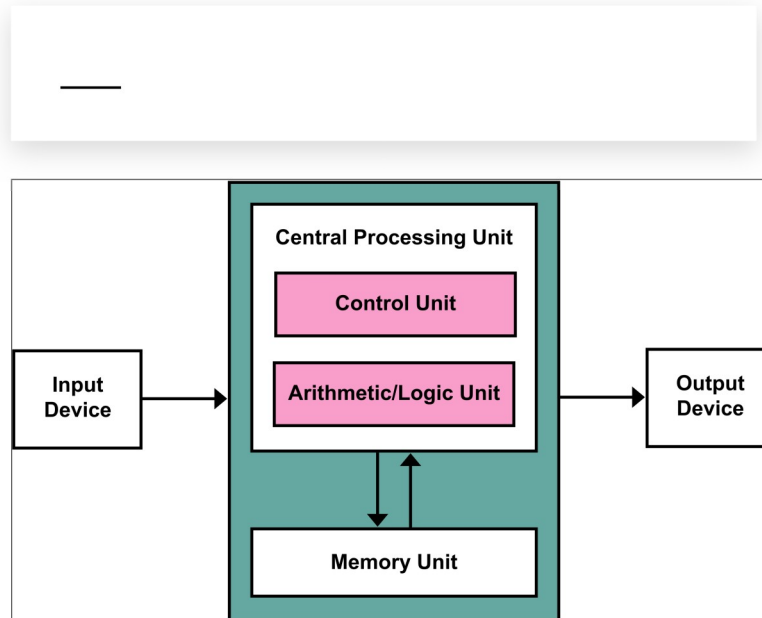
# I computer

- eseguono (poche) istruzioni semplici *velocissimamente*
- i dati e le istruzioni sono codificati come stringhe di bit 0/1
  - numeri interi o con la virgola (sono diversi!!!)
  - testi
  - strutture dati più complesse
    - **liste** di oggetti
    - **insiemi** di oggetti
    - **dizionari** di oggetti
    - ...

# L'architettura di Von

## Neumann

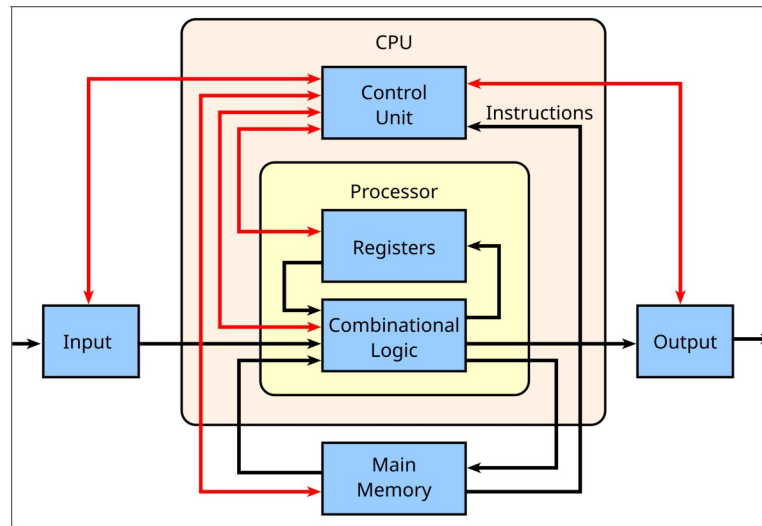
- **CPU** (Central Processing Unit) che fa i calcoli
- **RAM** (Random Access Memory) che contiene i dati (quando il PC è acceso)
- **BUS** di interconnessione tra le componenti
- Controllori delle periferiche di Input/Output
  - HD (Hard Disk) che contiene i programmi ed i file (anche quando il PC è spento)



- Scheda video
- Scheda di rete
- Tastiera ...

# CPU (Central Processing Unit)

- **CU** (Control Unit) decodifica l'istruzione e coordina le unità funzionali per eseguirla
- **ALU** (Aritmetic Logic Unit) unità funzionale che esegue le operazioni sui dati
- **PC** (Program Counter) indica l'indirizzo in memoria della prossima istruzione da eseguire
- **Registri**: piccolo spazio di memoria veloce per i valori intermedi dei calcoli





# Python (da Monty Python)

creato da Guido Van Rossum

- nato come linguaggio di scripting del sistema operativo Amoeba
- progettato per essere **molto leggibile** e **facile da imparare**
- l' **indentazione** (allineamento dell'istruzione) fa parte della sintassi
- (finalmente) le parentesi graffe e i punti e virgola di C/C++ e Java scompaiono
- ci sono molti costrutti facili da leggere (e.g. list comprehension)



## Primi concetti fondamentali

- le **informazioni** si trovano nella **memoria**
- le **variabili** sono dei **nomi** che corrispondono a **spazi/posizioni** in memoria che contengono i dati
- le **istruzioni** indicano al PC quali operazione compiere
- le **funzioni/procedure** sono parti di codice riusabili
  - le **funzioni** restituiscono un risultato
  - le **procedure** NON restituiscono un risultato  
(ma fanno cose)

## Tipi di dati: interi (di tipo `int`)

sono codificati in memoria come sequenza di cifre binarie (0/1)

MA a precisione "infinita"!

Python alloca (riserva) automaticamente tutto lo spazio di memoria necessario a contenere la codifica binaria del numero

Il linguaggio **C/C++** invece usa una word (64 bit su un PC a 64 bit) e può rappresentare al massimo il valore  $2^{63} - 1 = 9\_223\_372\_036\_854\_775\_807$  (circa 9 miliardi di miliardi)

Com'è codificato un numero naturale in binario

- in base 2, ovvero usando solo le cifre 0 e 1
- come per i numeri in base 10, ciascuna cifra in posizione  $P = 0, 1, 2, 3 \dots$   
(contando da destra) 'pesa'  $base^P$

|                  |          |          |          |          |
|------------------|----------|----------|----------|----------|
| <b>Cifre</b>     | <b>0</b> | <b>1</b> | <b>1</b> | <b>0</b> |
| <b>Posizione</b> | 3        | 2        | 1        | 0        |
| <b>Peso...</b>   | $2^3$    | $2^2$    | $2^1$    | $2^0$    |
| <b>...ovvero</b> | 8        | 4        | 2        | 1        |

**Contributo**     $8 * 0$      $4 * 1$      $2 * 1$      $1 * 0$

- Risultato:  $0 + 4 + 2 + 0 = 6$
- il più grande numero con 4 cifre è 1111 ovvero  
 $8 + 4 + 2 + 1 = 15 = 2^4 - 1$



Com'è codificato un numero relativo (col segno) in binario

La codifica più comune viene chiamata "in complemento a 2"

I pesi sono gli stessi, ma il peso del bit più a sinistra (il più significativo) ha valore negativo

| Cifre     | 1        | 1     | 1     | 0     |
|-----------|----------|-------|-------|-------|
| Posizione | 3        | 2     | 1     | 0     |
| Peso      | $-(2^3)$ | $2^2$ | $2^1$ | $2^0$ |
| ovvero    | -8       | 4     | 2     | 1     |

**Contributo**     $-8 * 1$      $4 * 1$      $2 * 1$      $1 * 0$

- Risultato:  $-8 + 4 + 2 + 0 = -2$
- Il più piccolo numero di 4 bit sarà  $1000 = -8 + 0 + 0 + 0 = -8 = -(2^3)$
- Il più grande sarà  $0111 = 0 + 4 + 2 + 1 = 7 = 2^3 - 1$
- **Intervallo di rappresentazione per  $n$  cifre:**

$$[-2^{n-1}, 2^{n-1} - 1]$$

In [3]:

[illegible]

Out[3]:

10000000000000000000000000000000000000000000000000000000

In [ ]:

```
# NOTA: per sapere un dato di che tipo è si usa la funzione 'type'
type(1_000_000_000_000_000_000_000_000_000_000_000_000_000_000)
```

## Operazioni matematiche

Sono presenti le solite 4 operazioni  $+$ ,  $-$ ,  $*$ ,  $/$ , e inoltre

- `//` : la divisione intera  
(che produce **int** solo se entrambi gli operandi sono **int**)  
NB: la divisione normale `/` produce sempre **float**
- `%` : il resto della divisione intera (detto anche modulo)
- `**` : la potenza

Altre funzioni matematiche sono disponibili nella libreria `math`

In [ ]:

```
10**1000 # '**' vuol dire 'elevato a potenza' -> (1 seguito da 1000 zeri)
```

In [ ]:

```
# divisione intera con //  
print('la divisione intera 81 // 7 fa', 81 // 7)
```

In [ ]:

```
# resto della divisione con %  
print('il resto di 81 // 7 è', 81 % 7)
```

In [ ]:

```
# verificiamo  
print('infatti 4+7*11 fa', 4+7*11)
```

## Numeri con la virgola (di tipo `float` )

Sono codificati con un numero di bit fisso (1 o 2 word)

(Vedi standard **IEEE 754**)

Esempio: word da 64 bit (per CPU a 64 bit)

| <b>segno (1 bit)</b> | <b>esponente (11 bit)</b> | <b>mantissa (52 bit)</b> |
|----------------------|---------------------------|--------------------------|
|----------------------|---------------------------|--------------------------|

0 = +

1 = -

esponente

cifre significative

Hanno un numero fisso di bit nella mantissa, quindi la precisione (cioè il numero di cifre corrette) è limitata

- una word -> 64 bit -> 52 bit di mantissa -> circa 16 cifre decimali
- due word -> 128 bit -> 112 bit di mantissa -> circa 33 cifre decimali

Esempio (con 1 bit di segno, 3 bit di esponente e 4 bit di mantissa)

| S | E | E | E | M | M | M | M |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |

- Il numero si trova calcolando

$$(-1)^{\text{segno}} * 2^{\text{esponente} + \text{offset}} * 1.\text{mantissa}$$

- segno = 1 -> numero negativo
- esponente = 010 -> in complemento a 2 vale 2
- per comodità supponiamo che l'offset sia 0 (dipende in realtà dallo standard)
- mantissa = 1011 -> viene aggiunta un 1 (e una virgola) a sinistra
  - 1.1011 viene convertito in decimale sapendo che i pesi, andando verso destra si dividono per 2 -> (0.1 = 1/2, 0.01 = 1/4, 1/8, 1/16, 1/32)

- quindi il valore della parte mantissa 1.1011 è

$$1 + 1/2 + 0 + 1/8 + 1/16 = 27/16 = 1.6875$$

- il numero quindi è:  $-1 * 2^{2+0} * 1.6875 = -6.75$

In [ ]:

```
# Vediamo quante cifre decimali possono essere rappresentate da una mantissa di 52 bit
# 2 alla 52 è maggiore di 10 alla 15 e minore di 10 alla 16?
print(' 52 bit sono circa 16 cifre decimali?', 10**15 < 2**52 < 10**16)
print(2**52, 10**15, 10**16)
```

In [ ]:

```
# 2 alla 112 è maggiore di 10 alla 33 e minore di 10 alla 34?
print('112 bit sono circa 33 cifre decimali?', 10**33 < 2**112 < 10**34)
```

In [ ]:

```
# vediamo che succede se stampo 10 alla 15 oppure 10 alla 16
print(10.0**15, 10.0**16) # del primo stampa TUTTE LE CIFRE! il secondo è invece "approssimato"

type(17.5) # ecco il tipo di un numero con la virgola
```

In [ ]:

```
# ESPRESSIONI MATEMATICHE
# posso scrivere espressioni matematiche e calcolarle
# (valgono le precedenze: PRIMA **, POI * e / E POI + e -)
print('3 + 14 * 5 / 4 = ', 3 + 14 * 5 / 4 )
# prima calcola 14*5=70 poi 70/4=17.5 poi 17.5+3=20.5
```

In [ ]:

```
print('3 + 5 * 10**2 / 4 = ', 3 + 5 * 10**2 / 4)
# prima calcola 10**2=100 poi 5*100=500 poi 500/4=125.0 poi 125.0+3=128.0
# NOTA la divisione produce un float e da quel momento in poi i risultati sono float
```

In [ ]:

```
# ma se voglio che vengano calcolate in ordine diverso
# basta usare le parentesi tonde
# (SOLO TONDE! Le quadre e graffe hanno significato diverso)
print('14 * 5 / (4 + 3) = ', 14 * 5 / (4 + 3))
# prima calcola 14*5=70 poi 4+3=7 poi 70/7=10
# dato che uso la divisione normale ottengo sempre un float
```

Valori vero/falso (di tipo `bool` )

Valori che indicano se il risultato di un confronto è vero o falso

- **True** : valore vero
- **False** : valore falso

Vengono usati nelle condizioni delle istruzioni di controllo che servono ad indicare al programma di eseguire parti diverse a seconda se la condizione è vera o falsa

Sono unici in tutto il programma, quindi facili da verificare

In [ ]:

```
# Per comodità di programmazione, se interpretati come bool,  
# alcuni valori valgono/significano "VERO", altri "FALSO"  
  
# il valore intero 0, se lo consideriamo come vero/falso viene considerato False,  
# tutti gli altri interi vengono considerati True  
bool(0), bool(1), bool(2), bool(-42)
```

In [ ]:

```
# che tipo hanno i due valori?  
type(True), type(False)
```

In [ ]:

```
# Anche per i float, 0.0 vale False mentre tutti gli altri float valgono True  
bool(0.0), bool(0.00000001), bool(0.5), bool(-3.14)
```

In [ ]:

```
# viceversa, se proviamo a trasformare True/False in int o float  
# i due valori True e False corrispondono  
# agli interi 1 e 0 e ai float 1.0 e 0.0  
int(True), int(False), float(True), float(False)
```

Il testo viene chiamato "stringa" (di tipo `str`)

E' formato da sequenze di caratteri

i testi sono **IMMUTABILI** (potete solo crearne di nuovi in memoria)

Ciascun carattere è codificato con la codifica **UNICODE** che contiene tutti i possibili caratteri (Ebraico, Greco, Arabo, Tamil, Geroglifico, Giapponese ...)

Si scrivono racchiusi tra singoli ( `'` ) o doppi apici ( `"` )

In [ ]:

```
# esempio di un testo  
"Paperino andò al mare"
```

Possiamo estrarre un carattere dalla stringa indicandone la posizione da sinistra (partendo da 0)

|              |   |   |   |   |   |   |          |   |   |          |
|--------------|---|---|---|---|---|---|----------|---|---|----------|
| Caratteri -> | " | P | a | p | e | r | <u>i</u> | n | o | "        |
| Posizione -> |   | 0 | 1 | 2 | 3 | 4 | <u>5</u> | 6 | 7 | (indice) |

In [ ]:

```
# estraggo il carattere in posizione 5 (partendo da 0)
A = "Paperino"
A[5]    # leggo il valore che sta all'indice 5
```

In [ ]:

```
# e ne calcolo il codice con 'ord' (e il tipo del dato contenuto in A)
ord(A[5]), ord('i'), type(A)
```

In [ ]:

```
# oppure posso ottenere il carattere a partire dal suo codice con 'chr'
chr(105)
```

In [ ]:

```
# Posso concatenare due o più stringhe ottenendone una nuova con '+'  
'Paperino' + ' ' + "andò al mare" + ' con Minnie'
```

In [ ]:

```
# o ripetere più volte la stessa concatenazione con '*'  
"Minnie " * 5
```

In [ ]:

```
# o spezzare la stringa rispetto a un separatore (per default gli spazi)  
# con il *metodo* 'split'  
A = 'Paperino andò al mare con Pippo e \n Minnie'  
A.split() # chiedo alla stringa di spezzarsi sugli spazi  
# ottengo una lista di frammenti (stringhe)
```

In [ ]:

```
# Alcuni caratteri sono speciali e possono essere indicati con backslash+carattere
accapo      = '\n'      # newline
tab         = '\t'      # tab
backslash   = '\\'
doppio_apice = '\"'
apice       = '\''
print('Paperino\\nandò\\tal\\tmare')
```

Testo su più righe

Si racchiude in tripli apici ' ' ' o tripli doppi apici " " "

Possiamo direttamente usare gli accapi dover inserire \n

```
'''Paperino va al mare  
con Minnie  
e Topolino'''
```

## Variabili (nome -> spazio in memoria)

Sono elencate nella "**tabella dei nomi**" del programma (namespace)

Ad ogni "nome" corrisponde un pezzetto di memoria

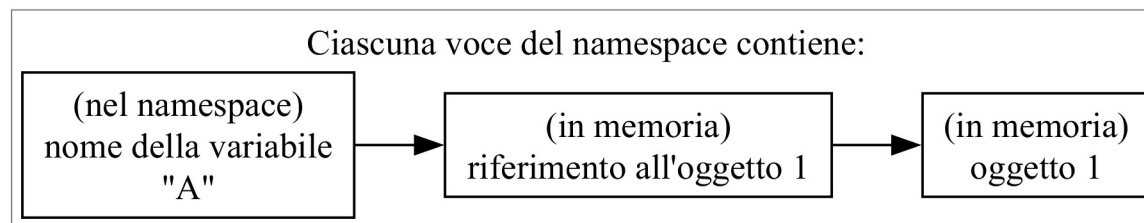
Che a sua volta contiene il riferimento (indirizzo in memoria) dell'oggetto  
"contenuto" nella variabile

Si chiama "variabile" perchè il suo contenuto può cambiare durante  
l'esecuzione del programma

In [7]:

```
figura
```

Out[7]:



Perchè questo indirizzamento "indiretto"? Come si comporta un linguaggio compilato come il C?

In **C** la 'tabella dei nomi' esiste SOLO nel compilatore che decide una volta per tutte dove mettere le variabili in memoria

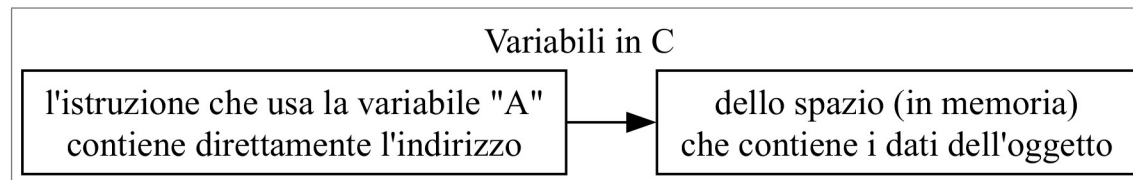
Nel programma eseguibile **i nomi delle variabili non esistono** e nel codice oggetto sono direttamente presenti solo le **posizioni degli spazi di memoria che contengono i dati**

Questo è molto efficiente (ma rigido)

In [10]:

```
figura2
```

Out[10]:





Allora perchè in Python c'è un doppio legame?

Nei linguaggi interpretati e non tipati come il Python si è scelto di permettere ad una variabile di **contenere un dato di tipo qualsiasi** anche in **momenti diversi dell'esecuzione**.

Quindi la tabella dei nomi contiene i riferimenti, che a loro volta indicano il dato (di tipo qualsiasi)

In [ ]:

```
figura
```

Come modificare una variabile?

Con l'operazione di assegnamento =

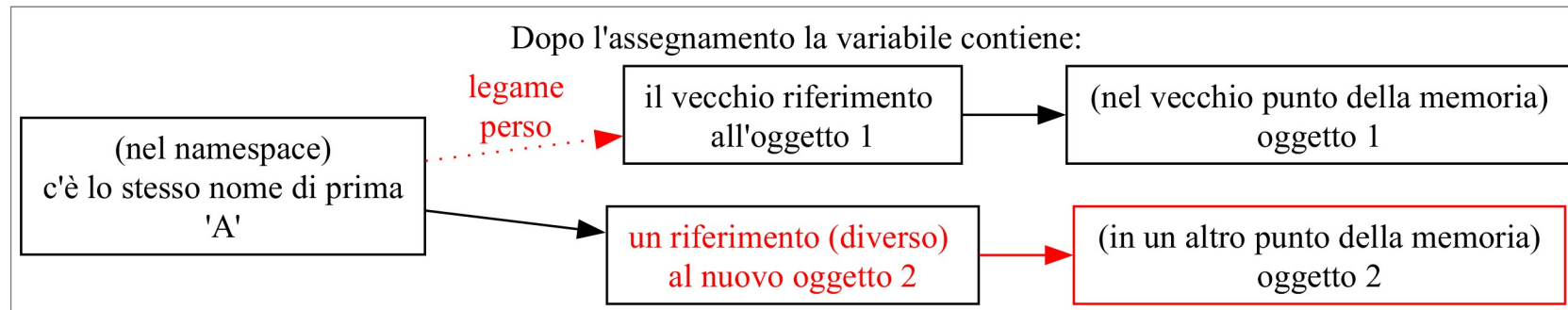
```
nome_variabile = espressione
```

L'assegnamento cambia il **riferimento** associato al **nome** nella **tabella dei nomi (namespace)**

In [14]:

```
# dopo aver assegnato un nuovo valore alla variabile A  
figura3
```

Out[14]:



In [1]:

```
# Come individuare il riferimento di un oggetto? con l'istruzione `id`  
A = 'Paperino'  
B = 'Minnie'  
print('A =', A, 'B =', B)  
print('id(A) =', id(A), 'id(B) =', id(B))
```

```
A = Paperino B = Minnie  
id(A) = 2493784302704 id(B) = 2493784234176
```

In [ ]:

```
A = B      # modifico la variabile A inserendoci il valore contenuto in B  
print('dopo aver eseguito A=B')  
print('A =', A, 'B =', B)  
print('id(A) =', id(A), 'id(B) =', id(B))  
# dopo l'assegnamento A contiene lo stesso riferimento di B  
# quindi lo stesso oggetto in memoria
```

In [ ]:

```
# due variabili "contengono" lo stesso identico oggetto se gli id sono uguali  
# possiamo controllarlo con l'operatore `is`  
print('A contiene lo stesso oggetto di B?', A is B)  
  
# vediamo se confronto A con un oggetto diverso  
C = 42  
print('A contiene lo stesso oggetto di C?', A is C)
```

In [ ]:

```
# due oggetti possono avere ID diversi ma contenere le stesse informazioni
A = 'Paperino '*4          # costruisco una stringa
B = 'Paperino '*4          # ne costruisco un'altra con lo stesso contenuto
print('A=',A)
print('B=',B)
```

In [ ]:

```
# due oggetti sono lo stesso oggetto in memoria? uso l'operatore 'is'
print('A contiene lo stesso esatto oggetto di B?', A is B)
# due oggetti sono simili? uso l'operatore '==' (doppio uguale)
print('A contiene le stesse informazioni di B?', A == B)
```

Operatori di assegnamento 'potenziato'

basta precedere = con l'operatore matematico

Può far comodo modificare una variabile aggiungendole, sottraendole, moltiplicandola o dividendola per un valore

- $A += 1$  **incremento** (come  $A = A+1$  ma più compatto)
- $A -= 2$  **decremento** (come  $A = A-2$  ma più compatto)
- $A *= 3$  **moltiplicazione** (come  $A = A*3$  ma più compatto)
- $A /= 4$  **divisione** (come  $A = A/4$  ma più compatto)

In [ ]:

```
# Esempio
A = 42
print(A)
A += 1
print('più 1', A)
A -= 12
print('meno 12', A)
A *= 10
print('per 10', A)
A /= 5
print('diviso 5', A)
```

In [ ]:

```
# Se applicati alle stringhe, gli assegnamenti potenziati '+=' e '*='  
# eseguono concatenazioni o ripetizioni  
A = 'Paperoga '  
print(A)  
A += 'e Gastone | '  
print(A)  
A *= 3  
print(A)
```

| Nei confronti tra oggetti, cosa usare?                        | Operatore          |
|---------------------------------------------------------------|--------------------|
| Sono esattamente lo stesso oggetto in memoria?                | <code>is</code>    |
| Contengono le stesse informazioni? sono simili?               | <code>==</code>    |
| Il primo è strettamente minore? (sta prima nell'ordinamento)  | <code>&lt;</code>  |
| Il primo è strettamente maggiore? (sta dopo nell'ordinamento) | <code>&gt;</code>  |
| Il primo è minore o uguale?                                   | <code>&lt;=</code> |
| Il primo è maggiore o uguale?                                 | <code>&gt;=</code> |

In [ ]:

```
# Posso confrontare numeri (anche di tipo diverso)  
print('12 < 11.3?', 12 < 11.3)
```

In [ ]:

```
# Posso confrontare stringhe rispetto all'ordinamento alfabetico  
print("'Paperino' < 'Topolino'?", 'Paperino' < 'Topolino')
```

In [ ]:

```
# MA NON POSSO confrontare numeri e stringhe  
print("42.17 < 'Topolino'?", 42.17 < 'Topolino')
```

In [ ]:

```
# A meno di non convertire prima il numero in una stringa  
print("str(42.17) < 'Topolino'?", str(42.17) < 'Topolino')
```

## Conversioni tra tipi diversi

- si usa il nome del tipo `tipo(valore da convertire)`

In [ ]:

```
# da numeri a stringhe  
# (voglio il testo che li rappresenta stampandoli come sequenza di caratteri)  
str(42), str(17.34)
```

In [ ]:

```
# da stringhe a numeri int o float  
# voglio il numero dalla sua rappresentazione testuale  
int('23'), int('-46'), float('23'), float('-46'), float('3.2415')
```

In [ ]:

```
# ATTENZIONE!  
# che succede se la stringa non rappresenta un numero di quel tipo?  
int('3.5')  
  
# ERRORE!!!! "Tradotto: il testo non rappresenta un intero in base 10"
```

In [ ]:

```
int('ABC')    # Errore: Tradotto: "in base 10 questo non è un numero"
```

In [ ]:

```
# ma se fornisco anche la base del numero contenuto nella stringa...
print(int('abc',16))
int('C', 16)   # in base 16 i numeri dopo il 9 sono rappresentati
               # dalle lettere A=10, B=11, C=12, D=13, E=14 e F=15
```

## Come leggere un dato con `input`

```
testo = input("Messaggio che chiede di inserire il dato (prompt)")
```

**NOTA: il risultato è SEMPRE una stringa** (che potete poi convertire in intero o float)

In [ ]:

```
# input di un testo  
nome = input("Come ti chiami? ")  
print("Mi chiamo", nome)
```

In [ ]:

```
# input di un intero  
testo = input("Quanto sei alto? (int) ")  
altezza = int(testo)  
print("Sono alto ", altezza, "cm")
```

In [ ]:

```
# input di un float  
testo = input("Quanto pesi? (float)")  
peso = float(testo)  
print("Peso ", peso, "kg")
```

# Esercizi per casa

## Tipi di Dati, Input, Stampa, Conversioni ed Espressioni

### Esercizio 1: Somma e Prodotto

Scrivi un programma che:

- Chiede di inserire due numeri interi
- Calcola e stampa la somma e il prodotto
- Stampa i risultati e i loro tipi

```
# Esempio di output:  
# Inserisci il primo numero intero: 5  
# Inserisci il secondo numero intero: 7  
# Somma: 12 (tipo: <class 'int'>)  
# Prodotto: 35 (tipo: <class 'int'>)
```

## Esercizio 2: Divisione normale, divisione intera e resto

Crea un programma che:

- Chiede di inserire due numeri interi
- Calcola la divisione normale, la divisione intera ed il resto del primo per il secondo
- Stampa i risultati e il loro tipo

```
# Esempio di output:  
# Inserisci un numero intero: 100  
# Inserisci un numero intero: 7  
# Divisione normale: 14.285714285714286 (tipo: <class 'float'> )  
# Divisione intera: 14 (tipo: <class 'int'> )  
# Resto: 2 (tipo: <class 'int'> )
```

## Esercizio 3: Manipolazione di Stringhe

Scrivi un programma che:

- Chiede di inserire una parola P ed un intero N
- Stampa la parola P ripetuta N volte con concatenazione
- Stampa la lunghezza della parola originale e il suo tipo

```
# Esempio di output:  
# Inserisci una parola: ciao  
# Inserisci un intero: 3  
# Ripetizione: ciaociaociao (tipo: <class 'str'>)  
# Lunghezza: 4 (tipo: <class 'int'>)
```

## Esercizio 4: Conversione e Espressione Mista

Crea un programma che:

- Chiede di inserire un numero come stringa
- Converte la stringa in intero e in float
- Calcola l'espressione: intero + float \* 2
- Stampa il risultato e il tipo

```
# Esempio di output:  
# Inserisci un numero: 3  
# Risultato di 3 + 3.0 * 2: 9.0 (tipo: <class 'float'>)
```

## Esercizio 5: Valore Booleano di Oggetti

Sviluppa un programma che:

- Chiede di inserire una stringa
- Converte la stringa in un valore booleano usando `bool()`
- Stampa il valore booleano e il suo tipo
- Spiega tramite commento quando `bool()` restituisce `False`

```
# Esempio di output:  
# Inserisci una stringa: Python  
# Valore booleano: True (tipo: <class 'bool'>)  
# Nota: bool() restituisce False SOLO se la stringa è vuota
```

## Note per gli Studenti:

- Utilizzate `input()` per leggere dati
- Convertite con `int()`, `float()`, `str()`, `bool()`
- Usate `print()` per visualizzare risultati e `type()` per verificare i tipi
- Usate `len()` per ottenere la lunghezza di un testo
- Sperimentate variando i valori di input